

Lecture 05 (bonus video)

Data Operations, Binary Operations, and Floating Point

Comp_Sci 213

Introduction to Computer Systems

Friday, Oct. 9, 2026

Kate Smith – Fall 2026

Slides adapted from:

Ghena, St-Amour, Hardavellas, Bustamente (Northwestern), Bryant, O'Hallaron (CMU), Garcia, Weaver (UC Berkeley)

Encoding strings (The C way)

```
char S[6] = "18243";
```

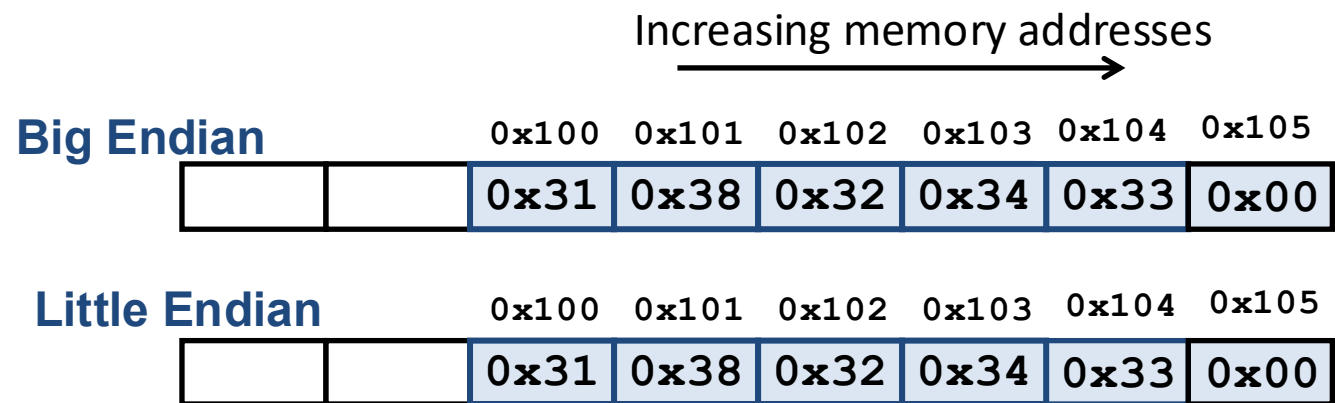
Dec	Hx	Oct	Html	Chr
48	30	060	0	0
49	31	061	1	1
50	32	062	2	2
51	33	063	3	3
52	34	064	4	4
53	35	065	5	5
54	36	066	6	6
55	37	067	7	7
56	38	070	8	8
57	39	071	9	9

Let's represent a number as a string! This is represented by array of characters

- Each character encoded in ASCII format
- NULL character (code 0) to mark the end
- The string has five characters, and the sixth byte holds the terminating NULL character ('\0', value 0, 0x00)
 - If `char S[] = "18243";` with no size, the compiler makes the array 6 bytes and includes the null automatically

Compatibility

1. Byte ordering not an issue (data all single-byte!) since endianness is for multi-byte values. Array elements always go in increasing address order
2. ASCII text files generally platform independent, except for different conventions of line termination character(s)!



Encoding strings, continued

- What happens if we store as an int rather than a character string?
 - $18243 = 0x4743$
- Endianness would be apparent!
 - Little endian: 43 47
 - Big endian: 47 43
- Integer encoding depends on byte order, but text form doesn't!

Topics

- Integer Operations
- Binary Operations – Boolean Algebra
- Bit Shifting
- Bit Masking
- Floating Point

Topics

- **Integer Operations**
- Binary Operations – Boolean Algebra
- Bit Shifting
- Bit Masking
- Floating Point

C versus the hardware

- Operations you can perform on types have edge conditions
 - Usually going above or below the bounds for the given size/encoding
- If we say what happens in that scenario, it'll be what “the hardware” (i.e., a computer) does
 - In today's examples, pretty much every computer does the same thing
- That is not the same as what C does
 - Unclear choices are left as **undefined behavior**
 - **Undefined Behavior:** C standard places no requirement on what happens if a program does that thing
 - The compiler can make any choice it wants
 - Program might crash, produce garbage, appear to work, behave differently based on optimization level, etc.

Unsigned Addition

- Like grade-school addition, but in base 2, and ignores final carry
 - If you want, can do addition in base 10 and convert to base 2. Same result! But here we're going to understand what the hardware is doing.
- **Example: Adding two 4-bit numbers**

$$\begin{array}{r} \overset{1}{0}\overset{1}{1}\overset{1}{0}1 \\ + \quad 0011 \\ \hline 1000 \end{array}$$

- $5_{10} + 3_{10} = 8_{10}$ ✓

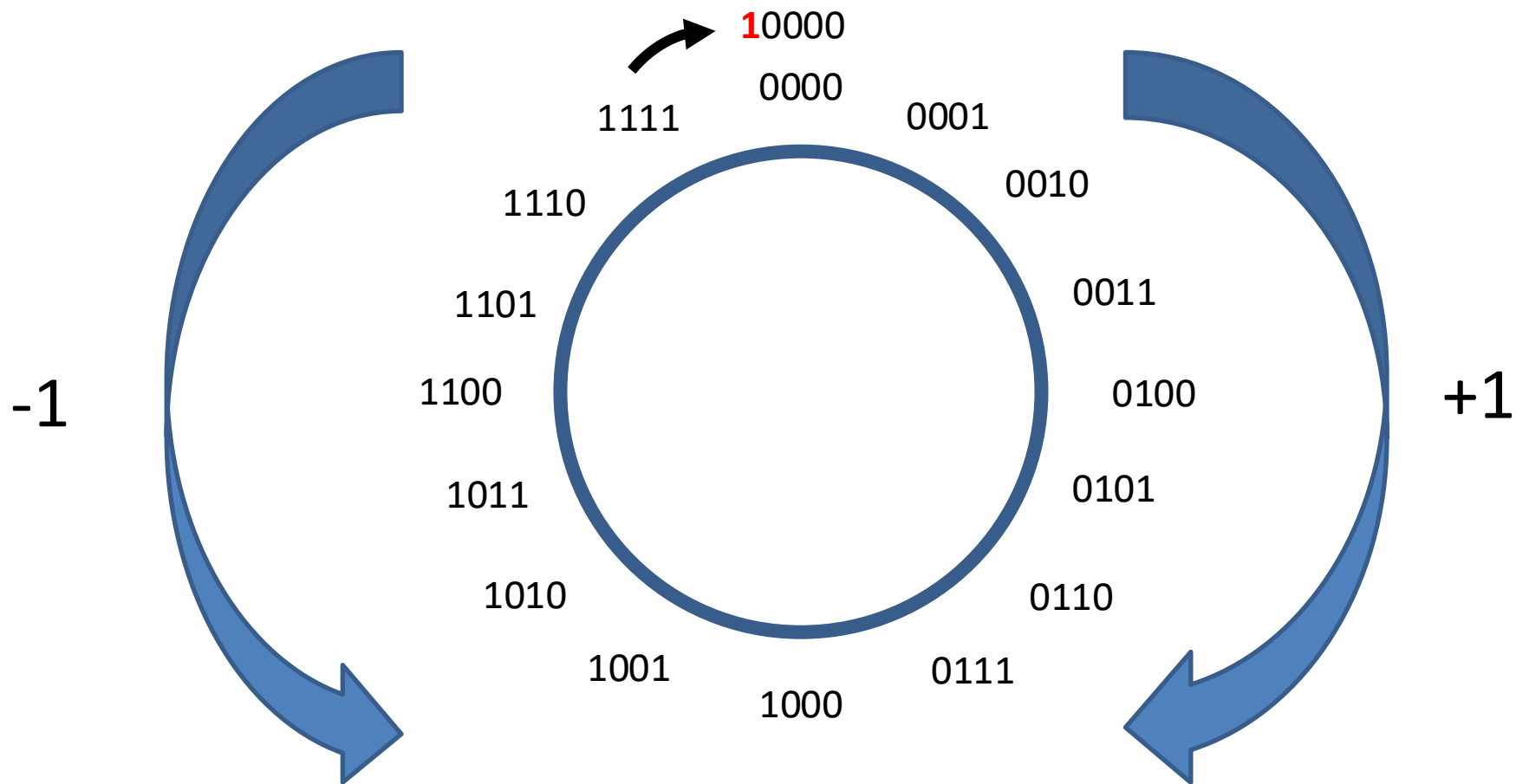
Unsigned Addition and Overflow

- What happens if the numbers get too big?
- **Example: Adding two 4-bit numbers**

$$\begin{array}{r} \overset{1}{1} \overset{1}{1} \overset{1}{1} \overset{1}{1} \\ 1101 \\ + 0011 \\ \hline 10000 \end{array}$$

- $13_{10} + 3_{10} = 16_{10}$
 - Too large for 4 bits!
 - Truncate most-significant bits that do not fit
 - Gives us modular (= modulo) behavior: $16 \text{ modulo } 2^4 = 0$
 - Final result is the 4 least significant bits (all we can fit): so 0_{10}

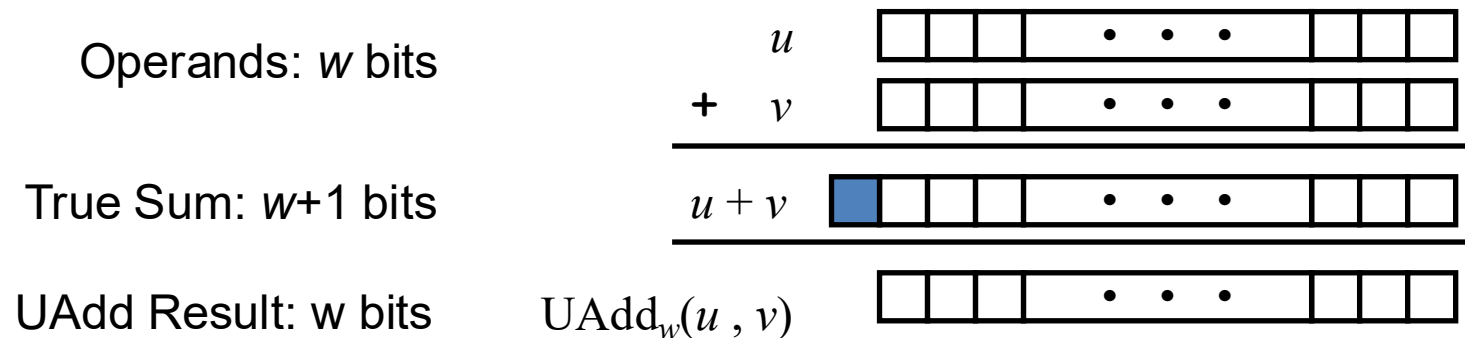
Modulo behavior in binary numbers



****Note:**
 $N \bmod r$ ($N \% r$) will be
some value $[0, r-1]$

Unsigned addition is modular

- Implements modular arithmetic
 - $\text{UAdd}_w(u, v) = (u + v) \bmod 2^w$
- Need to drop carry-out bit, otherwise results will keep getting bigger
 - Example in base 10: $80_{10} + 40_{10} = 120_{10}$ (2-digit inputs become a 3-digit output!)



- Warning: C does not tell you that the result was truncated!
 - **Unsigned** addition in C silently truncates most-significant bits beyond the limit

Signed (2's Complement) Addition

- Works exactly the same as unsigned addition!
 - Just add the numbers in binary, and the result will work out
- Signed and unsigned sum have the exact same bit-level representation
 - Computers use the same machine instruction and the same hardware!
 - That's a big reason 2's complement is so nice! Shares operations with unsigned

Signed addition example

- Same addition method as unsigned
- **Example: Adding two 4-bit signed numbers**

$$\begin{array}{rcl} & \overset{1}{1} & \overset{1}{} \\ & 10\bar{1}1 & (-5) \\ + & 0011 & (+3) \\ \hline & 1110 & (-2) \end{array}$$

○ $-5_{10} + 3_{10} = -2_{10} \checkmark$

Combining negative and positive numbers

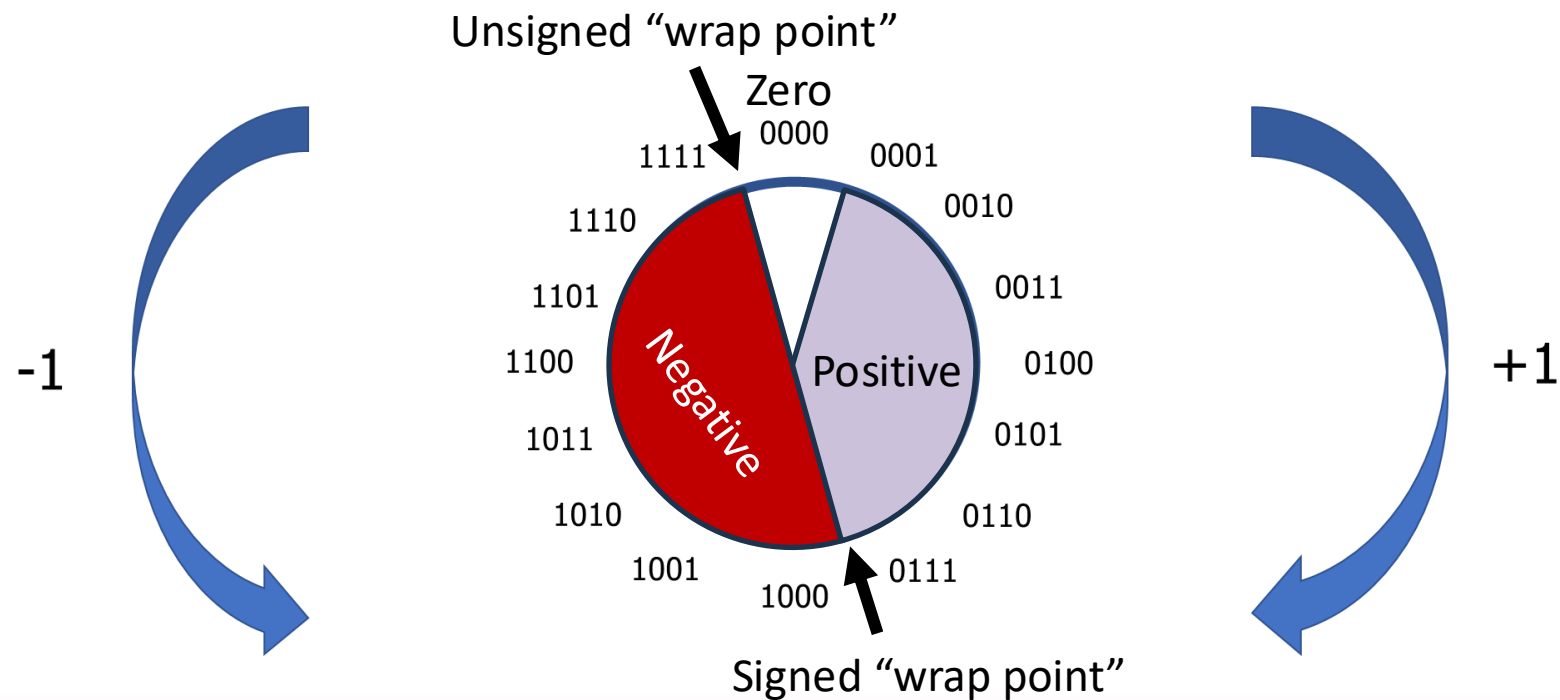
- Truncation makes signed addition work!
- **Example: Adding two 4-bit signed numbers**

$$\begin{array}{r} \overset{1}{1} \overset{1}{1} \overset{1}{1} \overset{1}{1} \\ 1101 \quad (-3) \\ + 0011 \quad (+3) \\ \hline 10000 \end{array}$$

- $-3_{10} + 3_{10} = 0_{10}$
 - Too large for 4 bits! Drop the carry bit
 - Result is what we expect as long as we truncate

Overflow of signed values

- Addition can lead to **overflow**
 - Unsigned: Result wraps around from big to small or vice versa
 - Signed: Both operands are positive and result is negative
 - Signed: Both operands are negative and result is positive



Signed addition and overflow

- Overflow can still happen in signed addition though
- **Example: Adding two 4-bit signed numbers**

$$\begin{array}{r} \overset{1}{0}\overset{1}{1}\overset{1}{0}1 \\ + \quad 0011 \\ \hline 1000 \end{array}$$

****For a fixed width w:**

- *Smallest value, most negative is 1 followed by all 0s bit pattern ($100\dots0 = -2^{w-1}$)*
- *Largest value, most positive is 0 followed by all 1s bit pattern ($01\dots1 = 2^{w-1} - 1$)*

- $5_{10} + 3_{10} = -8_{10}$ (+8 is too big to fit in 4 bits using a signed representation)
- Remember, this was also unsigned $5_{10} + 3_{10} = 8_{10}$

Signed addition and negative overflow

- Overflow also happens in the negative direction
- **Example: Adding two 4-bit signed numbers**

$$\begin{array}{r} \overset{1}{1} \overset{1}{1} \overset{1}{1} \\ 1011 \\ + 1011 \\ \hline 10110 \end{array}$$

****For a fixed width w:**

- *Smallest value, most negative is 1 followed by all 0s bit pattern ($100\dots0 = -2^{w-1}$)*
- *Largest value, most positive is 0 followed by all 1s bit pattern ($01\dots1 = 2^{w-1} - 1$)*

- $-5_{10} + -5_{10} = +6_{10}$ (-10 was too small to fit in 4 bits using a signed representation)

Overflow: hardware vs C standard

- Hardware implementations for unsigned and signed addition are the same
 - Both implement truncation of overflowing bits, leads to modular arithmetic
- Unsigned overflow in C is defined as modular arithmetic
 - Safe to use. Big number wraps around to a small number
- Signed overflow in C is **undefined behavior**
 - Compiler *probably* does modular result
 - But there are no promises about this, and it can make *assumptions*
 - So don't rely on it
 - Generally: overflow is bad and is to be avoided!

Negating a number

- In C:
 - $x = -y;$
- Operation
 - Determine the negative, signed version of the number (two's complement)
 - Hardware method: flip bits and add one
- Complement operator (~)
 - This is a bit-level operator
 - Flips all bits: zeros become a one and ones become a zero
 - $\sim 0b1011 \rightarrow 0b0100$

Negating via Complement & Increment

- Claim: The following is true for 2's complement

- $\sim x + 1 == -x$

$$\begin{array}{r}
 \mathbf{x} \quad \boxed{1} \boxed{0} \boxed{0} \boxed{1} \boxed{1} \boxed{1} \boxed{0} \boxed{1} \\
 + \quad \sim \mathbf{x} \quad \boxed{0} \boxed{1} \boxed{1} \boxed{0} \boxed{0} \boxed{0} \boxed{1} \boxed{0} \\
 \hline
 -1 \quad \boxed{1} \boxed{1} \boxed{1} \boxed{1} \boxed{1} \boxed{1} \boxed{1} \boxed{1}
 \end{array}$$

- Complement

- Observation: $\sim x + x == 1111...11_2 == -1$

- Increment

- $\sim x + 1 == \sim x + x - x + 1 == -1 - x + 1 == -x$

- Example, 4 bits: $6_{10} = 0110_2$

- Complement: $1001_2 \rightarrow$ Increment = $1010_2 = -8 + 2 = -6_{10}$

****Note: $x - x = 0$**

Subtraction in two's complement

- Subtraction becomes addition of the negative number
 - $5 - 3 = 5 + -3 = 2$
- Both unsigned and signed subtraction
 - Convert subtrahend to its two's complement negative form
 - i.e., negate it
 - Then do addition
 - Treat result as original encoding

$$\begin{array}{r} \overset{1}{0} \overset{1}{1} \overset{1}{0} 1 \quad (+5) \\ + \quad 1 1 0 1 \quad (-3) \\ \hline 1 0 0 1 0 \end{array}$$

C rules vs hardware rules

- Exact same overflow rules apply
- Unsigned subtraction overflow can wrap below zero to make a large number
 - Modular arithmetic
- Signed subtraction overflow is **UNDEFINED BEHAVIOR**
 - And therefore should not be trusted

Practice

- Adding two 8-bit binary numbers:
 - Also determine the decimal version of the result
 - Omitting the encoding is intentional

$$\begin{array}{r} 00010101 \\ + 10110001 \\ \hline \end{array}$$

Practice

- Adding two 8-bit binary numbers:
 - Also determine the decimal version of the result
 - Omitting the encoding is intentional

	Unsigned encoding		Signed encoding
$\begin{array}{r} \overset{1}{0}\overset{1}{0}\overset{1}{0}101 \\ + 10110001 \\ \hline 11000110 \end{array}$	$\begin{array}{l} 16+4+1 = 21 \\ 128+32+16+1 = 177 \\ 128+64+4+2 = 198 \end{array}$	OR	$\begin{array}{l} 16+4+1 = 21 \\ -128+32+16+1 = -79 \\ -128+64+4+2 = -58 \end{array}$

Practice

- Adding two 8-bit binary numbers:
 - Also determine the decimal version of the result
 - Omitting the encoding is intentional

	Unsigned encoding		Signed encoding
$\begin{array}{r} 111 \\ 00010101 \\ + 10110001 \\ \hline 11000110 \end{array}$	$16+4+1 = 21$ $128+32+16+1 = 177$ $128+64+4+2 = 198$	OR	$16+4+1 = 21$ $-128+32+16+1 = -79$ $-128+64+4+2 = -58$

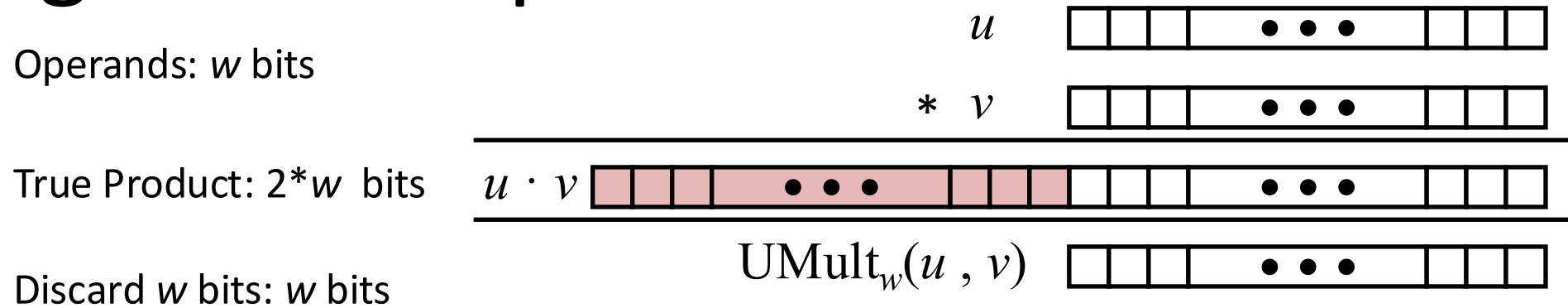
What about unsigned subtraction 21-79?

That would treat the result as unsigned, with the value 198. Modular arithmetic in action!!

Multiplication

- Goal: Compute the Product of two **w**-bit numbers x, y
 - Either signed or unsigned
- But, exact results can be bigger than **w** bits
 - Double the size ($2w$), in fact!
 - Example in base 10: $50_{10} * 20_{10} = 1000_{10}$
 - (2-digit inputs become a 4-digit output!)
- As with addition, result is truncated to fit in **w** bits
 - Because computers are finite, results can't grow indefinitely

Unsigned Multiplication



- **Standard Multiplication Function**
 - Equivalent to grade-school multiplication
 - But ignores most significant w bits of the result
 - As a person, we can do base 10 multiplication, convert to base 2, then truncate
- Implements modular arithmetic like addition does

$$\text{UMult}_w(u, v) = (u \cdot v) \bmod 2^w$$

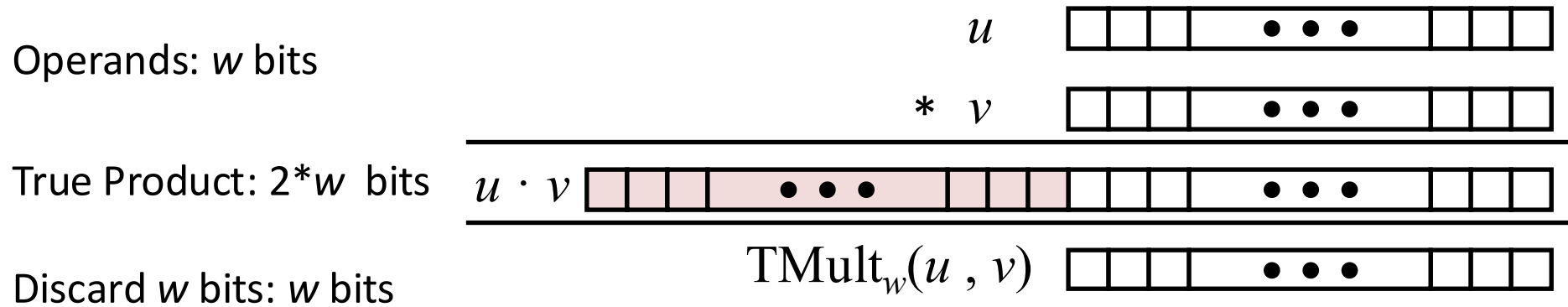
Unsigned multiplication

- Example: Multiplying two 4-bit numbers

$$\begin{array}{r} 0010 \\ \times 0101 \\ \hline 0010 \\ 00000 \\ 001000 \\ + 0000000 \\ \hline \textcolor{red}{000}1010 \end{array}$$

$$2_{10} * 5_{10} = 10_{10} \checkmark$$

Signed (2's Complement) Multiplication



- **Standard Multiplication Function** (same as unsigned)
 - Ignores most significant w bits
 - Lower bits still give the correct result
 - So we can use same machine instruction for both!
 - Again, that's one reason why 2's complement is so nice
- **In C, signed overflow is undefined**
 - ...but probably you'll see the two's complement behavior

Signed multiplication

- **Example: Multiplying two's complement 5-bit numbers**

$$\begin{array}{r} 11110 \quad -2 \\ \times 00011 \quad 3 \\ \hline 11110 \\ + 111100 \\ \hline \textcolor{red}{10}11010 \end{array}$$

What are these two 5-bit numbers?

What is the result of this addition?

$$-2_{10} * 3_{10} = -6_{10} \checkmark$$

What about divide?

- Annoying operation, not going to discuss in this class
 - Similar to long division process
 - Tedious and complicated to get right
- There are some computers that don't have hardware support for division at all!!
- Important thing to remember is that integers don't have fractional parts
 - In C: $1/2 == 0$
 - We'll need a different encoding for fractional numbers: floating point

Topics

- Integer Operations
- **Binary Operations – Boolean Algebra**
- Bit Shifting
- Bit Masking
- Floating Point

Boolean algebra

- You've programmed with **and** and **or** in earlier classes
 - C and C++ logical operations: `&&` (and), `||` (or)
- **Boolean algebra is a generalization of that**
 - A mathematical system to represent logic (propositional logic)
 - 2 truth values: true = **1**, false = **0**
 - Operations: and **&**, or **|**, not (or complement) **~**
 - And others

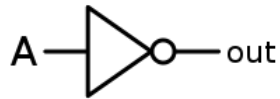
Aside: Propositions, logic, and mathematics

- Propositions affirm or denies a “predicate” of a “subject”
 - “All humans (subject) are mortal (predicate) -> True
 - “London (s) is a capitol city (p) AND London (s) is the capitol of Germany (p)” -> False
- Long ago, Aristotle and others studied logic in terms of rhetoric with philosophy
- In the 18th century, attempts were made to treat formal logic in a symbolic or algebraic way by philosophical mathematicians
 - Leibniz, Lambert
- In the 19th century, Boole and De Morgan developed the mathematical treatment of logic
 - Development of Boolean Algebra and important relationships within it
- In the 20th century, Claude Shannon applied Boolean logic to model the behavior of switching circuits
 - Shannon’s master’s thesis would establish the theory behind digital circuit-based computation!

Truth tables for Boolean algebra

- For each possible value of each input, what is the output
 - Column for each input
 - Column for the output operation

Complement (NOT)



$\sim A$

A	$\sim A$
0	1
1	0

OR



A | B

A	B	A B
0	0	0
0	1	1
1	0	1
1	1	1

AND



A & B

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

Performing Boolean algebra

- **Follow the rules for each operation to compute results**

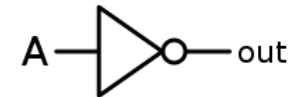
- Rules are like those you know from programming



OR: |



AND: &



NOT: ~

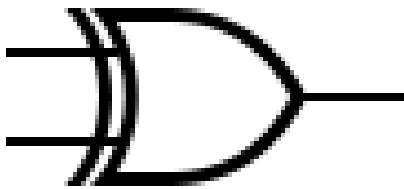
1: True 0: False

$$(1 \mid 0) \& 0 \longrightarrow 1 \& 0 \longrightarrow 0$$

$$(1 \& 1) \& \sim(0 \mid 0) \longrightarrow 1 \& \sim(0) \longrightarrow 1 \& 1 \longrightarrow 1$$

Exclusive Or (xor)

$A \wedge B$		
A	B	$A \wedge B$
0	0	0
0	1	1
1	0	1
1	1	0



- An operation you likely haven't used before:
 - Xor - either A or B, but not both
 - $\wedge$ symbol in C
- We can build Xor out of $\&$, $|$, and $\sim$
 - $A \wedge B = (\sim A \& B) | (A \& \sim B)$
 - (exactly one of A and B is true)
 - $A \wedge B = (A | B) \& \sim(A \& B)$
 - (either is true but not both are true)
 - The two definitions are equivalent
 - Produce the same Truth Table

Practice problem

		(A & B) B	
A	B	(A&B) B	
0	0		
0	1		
1	0		
1	1		

Practice problem

(A & B) B		
A	B	(A&B) B
0	0	0
0	1	1
1	0	0
1	1	1

Practice problem

(A & B) B		
A	B	(A&B) B
0	0	0
0	1	1
1	0	0
1	1	1

This is equivalent to B
(A has no influence on the solution)

De Morgan's Law

- We can express Boolean operators in terms of the others
- De Morgan's laws: allow swapping & with |
 - $A \& B = \sim(\sim A \mid \sim B)$ $\rightarrow$ $\sim(A \& B) = \sim A \mid \sim B$
 - (neither A nor B is false)
 - $A \mid B = \sim(\sim A \& \sim B)$ $\rightarrow$ $\sim(A \mid B) = \sim A \& \sim B$
 - (A and B are not both false)
 - Useful for simplifying logical statements

Note: Distributive property in Boolean algebra

- Boolean operation $\&$ distributes over $|$
 - $A \& (B | C) = (A \& B) | (A \& C)$
- Boolean operation $|$ also distributes over $\&$
 - $A | (B \& C) = (A | B) \& (A | C)$

This is different from what we typically assume because we cannot say that $A + (B * C) = (A * B) + (A * C)$ holds for all integers!

Generalized Boolean algebra

- Boolean operations can be extended to work on collections of bits (i.e., bytes)

- Operations are applied one bit at a time: **bitwise**

01101001	01101001	01101001	
<u>& 01010101</u>	<u> 01010101</u>	<u>^ 01010101</u>	<u>~ 01010101</u>
01000001	01111101	00111100	10101010

- All of the properties of Boolean algebra still apply
 - Relationships between operations, etc.
- Bitwise operations are usable in C: **&, |, ~, ^**
 - Can operate on any integer type (long, int, short, char, signed or unsigned)

Warning: bitwise operations are NOT logical operations

- Logical operations in C: `||`, `&&`, `!` (logical Or, And, and Not)
 - Only operate on a single bit
 - View 0 as “False”
 - View *anything nonzero* as “True”
 - Always return 0 or 1
 - Short-circuit evaluation: only checks the first operand if that is sufficient
- Examples
 - `!0x41 -> 0x00` `!0x00 -> 0x01` `!!0x41 -> 0x01`
 - `0x59 && 0x35 -> 0x01`
 - `(p != NULL) && *p` (short circuit evaluation avoids null pointer access)
- Don't confuse the bitwise and logical operations!!
 - This is a common C mistake

Practice

1. $0x37 \mid 0xAA$ 2. $0x06 \wedge 0xFF$

A B		
A	B	A B
0	0	0
0	1	1
1	0	1
1	1	1

A ^ B		
A	B	A & B
0	0	0
0	1	1
1	0	1
1	1	0

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Practice

1. $0x37 \mid 0xAA$

```
  00110111
| 10101010
-----
 10111111
```

A B		
A	B	A B
0	0	0
0	1	1
1	0	1
1	1	1

2. $0x06 \wedge 0xFF$

```
  00000110
^ 11111111
-----
 11111001
```

A ^ B		
A	B	A ^ B
0	0	0
0	1	1
1	0	1
1	1	0

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Practice: C example of bitwise operators

```
unsigned char x = 13;  
unsigned char y = 11;  
unsigned char z = x & y;
```

- What decimal value is in z now?
 - Remember: `unsigned char` is an 8-bit value

Practice: C example of bitwise operators

```
unsigned char x = 13;  
unsigned char y = 11;  
unsigned char z = x & y;
```

- What decimal value is in z now?
 - Remember: `unsigned char` is an 8-bit value
 - x: 0b00001101
 - y: 0b00001011
 - z: 0b00001001 -> 9

Topics

- Integer Operations
- Binary Operations – Boolean Algebra
- **Bit Shifting**
- Bit Masking
- Floating Point

Left Shift: $x \ll y$

- Shift bit-vector x left by y positions
 - Truncate back to maximum size
 - Throws away bits on the left
 - Fill empty bits with 0
 - Same behavior for signed or unsigned
- Equivalent to multiplying by 2^y
 - And then taking modulo (i.e. truncating overflow bits)
- Undefined behavior in C when:
 - $y < 0$, or $y \geq \text{bit_width}(x)$
 - Also when some non-0 bits get shifted off (*probably* they get truncated)

Argument x	00000010
$\ll 3$	000 00010 <u>000</u>

Argument x	10100010
$\ll 3$	101 00010 <u>000</u>

Right Shift: $x \gg y$

- Shift bit-vector x right y positions
 - Throw away extra bits on right
- But how to fill the new bits that open up?
 - Will depend on signed vs unsigned
- Unsigned: Logical shift
 - Always fill with 0's on left
- Signed: Arithmetic shift
 - Replicate most significant bit on left
 - Necessary for two's complement integer representation (sign extension!)
- Undefined behavior in C when:
 - $y < 0$, or $y \geq \text{bit_width}(x)$

Argument x	<u>0</u> 1100010
Logi. $\gg 2$	<u>00</u> 011000
Arith. $\gg 2$	<u>00</u> 011000

Argument x	<u>1</u> 0100010
Logi. $\gg 2$	<u>00</u> 101000
Arith. $\gg 2$	<u>11</u> 101000

Practice shifting in C

```
unsigned char x = 0b10100010;
```

```
x << 3 = ? 0b00010000
```

Steps:

0b10100010000

0b~~101~~00010000

```
unsigned char x = 0b10100010;
```

```
x >> 2 = ? 0b00101000
```

Steps:

0b0010100010

0b00101000~~10~~

```
signed char x = 0b10100010;
```

```
x >> 2 = ? 0b11101000
```

Steps:

0b1110100010

0b11101000~~10~~

Note:

GCC supports the prefix `0b` for binary literals (like `0x...` for hex) directly in C. This is not part of the C standard! It may not work on other compilers.

Concept: Not all operations are equally expensive!

- Some operations are pretty simple to perform in hardware
 - E.g., addition, shifting, bitwise operations
 - Also true of doing the same by hand on paper
- Others are much more involved
 - E.g., multiplication, or even more so division
 - Consider long multiplication / long division; quite tedious!
 - Hardware is not doing the exact same thing, but similar principle
- For best performance: swap expensive operations with simple ones!
 - Doesn't work in all cases, but often does when mult/div by constants
 - Don't do this yourself: the compiler will do it for you automatically

Shift to divide

- Division by powers of two could be shifts
 - `unsigned int x = y / 2;` `unsigned int x = y >> 1;`
 - Example `0b00100 >> 1 = 0b00010`
 - `4 -> 2`
- Even more important because division is a complicated operation
 - Multiply is implemented in (relatively) simple hardware on most systems
 - Compiler might actually translate your divide-by-powers-of-two operations into shift operations though!

Shifts for large values of k

- For data consisting of w bits, what is the effect of shifting by some value $k \geq w$? Assume $w = 32$ in the following.

```
int          lval = 0xFEDCBA98 << 32;  
int          aval = 0xFEDCBA98 >> 36;  
unsigned     uval = 0xFEDCBA98u >> 40;
```

- C standards don't specify what should be done (Undefined Behavior)
- In most cases, however, machines will compute the shift as $k \bmod w$:

```
lval  0xFEDCBA98 // shift of 0  
aval  0xFFEDCBA9 // shift of 4  
uval  0x00FEDCBA //shift of 8
```

- This behavior is not guaranteed! Best practices avoid shift amounts greater than w

Topics

- Integer Operations
- Binary Operations – Boolean Algebra
- Bit Shifting
- **Bit Masking**
- Floating Point

Bit Masking

- How do you manipulate certain bits within a number?
- Combines some of the ideas we've already learned
 - $\sim$, $\&$, $|$, $\ll$, $\gg$
- Steps
 1. Create a "bit mask" which is a pattern to choose certain bits
 2. Use $\&$ or $|$ to combine it with your number
 3. Optional: Use $\gg$ to move the bits to the least significant position

How to operate on bits

- Selecting bits, use the AND operation
 - 1 means to select that bit
 - 0 means to not select that bit
- Writing bits
 - Writing a one, use the OR operation
 - 1 means to write a one to that position
 - 0 is unchanged
 - Writing a zero, use the AND operation
 - 0 means to write a zero to that position
 - 1 is unchanged

Select bottom four bits:

```
num & 0x0F
```

Set 6th bit to one:

```
num | (1 << 6)  
num | (0b01000000)
```

Clear 6th bit to zero:

```
num & (~ (1 << 6))  
num & (~ (0b01000000))  
num & (0b10111111)
```

Example: swap nibbles in byte

- Nibble - 4 bits (one hexit)
 - Input: 0x4F -> Output 0xF4

- Method:

- 1. Shift and select upper four bits
- 2. Shift and select lower four bits
- 3. Combine the two nibbles

```
uint8_t lower = input >> 4;
```

```
uint8_t upper = input << 4;
```

```
uint8_t output = upper | lower; // combines two halves
```

What are the values of the new upper bits?

Unsigned -> Will be zero

Shifting implicitly zero'd out irrelevant bits.

Otherwise we would have needed an & operation too.

Example: selecting bits

Input: 0b01100100

- Select bits 2 and 3 from a number

Mask: 0b00001100

$$\begin{array}{r} 0b01100100 \\ \& \underline{0b00001100} \\ 0b00000100 \end{array}$$

Finally, shift right by two to get the values in the least significant position:

0b00000001

In C:

```
result = (input & 0x0C) >> 2;
```

Practice: determine if bit 7 is 1 or 0

- Need to select bit 7, and shift it to the least-significant position
 - Multiple ways to achieve this.

Practice: determine if bit 7 is 1 or 0

- Need to select bit 7, and shift it to the least-significant position
 - Multiple ways to achieve this.

```
bool result = (value & (1<<7)) >> 7;
```

- Grab the seventh bit, then shift it to the zero position

```
bool result = (value & 0b10000000) >> 7;
```

```
bool result = (value & 0x80) >> 7;
```

- Same as above, but hardcoded (and easier to make mistakes)

```
bool result = (value >> 7) & 1;
```

- Shift desired bit to the zero position, then select it

- ```
bool result = ((value & (1<<7)) != 0;
```

  - Select desired bit, then use logical operators to reduce to a single bit

*\*\*Note: it is best practice to be explicit and use parenthesis to group expressions!*

# Topics

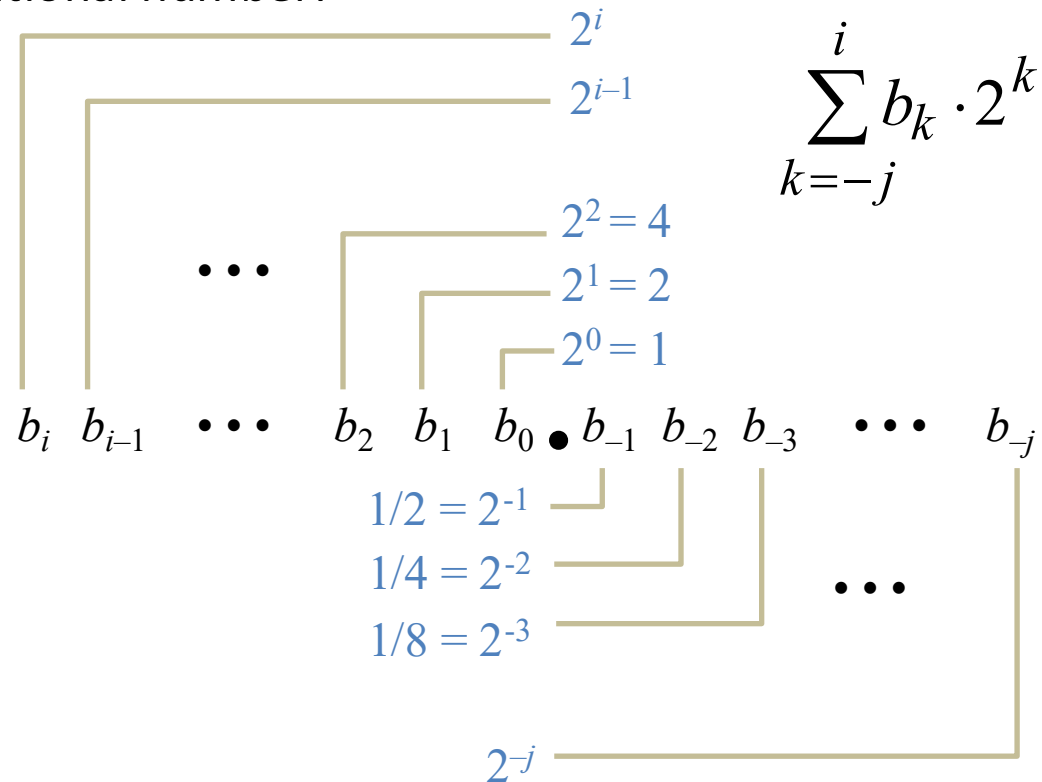
- Integer Operations
- Binary Operations – Boolean Algebra
- Bit Shifting
- Bit Masking
- **Floating Point**
  - **Fractional binary numbers**
  - Representing floating point

# Floating point numbers

- In decimal:
  - $123450_{10}$
  - $123.450_{10}$
  - $1.23450_{10}$
- We can use this same system in binary as well:
  - $1010110_2$  ( $86_{10}$ )
  - $1010.110_2$  ( $10.75_{10} = \frac{86}{2^3}$ )
  - $1.010110_2$  ( $1.34375_{10} = \frac{86}{2^6}$ )

# Fractional Binary Numbers

- Representation
  - Bits to right of “binary point” represent fractional powers of 2
  - Represents rational number:



# Example binary conversion

1010.110

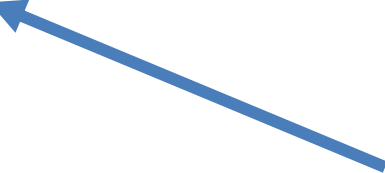
Before the binary point:

$$1*2^3 + 0*2^2 + 1*2^1 + 0*2^0 = 1*2^3 + 1*2^1 = 8+2 = 10$$

After the binary point:

$$1*2^{-1} + 1*2^{-2} + 0*2^{-3} = 1*2^{-1} + 1*2^{-2} = \frac{1}{2} + \frac{1}{4} = \frac{3}{4} = 0.75$$

# Fractional Binary Number Examples

- $5 + 3/4$  = 0b101.11  


Note:  
This the number 3!
- $2 + 7/8$  = 0b10.111  


This is the number 7!
- $63/64$  = 0b0.111111  


This is the number 63!

# Scientific Notation Binary

- Scientific notation works in binary just like in decimal
  - Decimal:  $134 = 1.34 * 10^2$
  - Binary:  $101 = 1.01 * 2^2$
- Positive and negative exponents change the direction the point moves
  - Positive powers get bigger:  $1.00101 * 2^4 \rightarrow 10010.1$
  - Negative powers get smaller:  $1.00101 * 2^{-3} \rightarrow 0.00100101$

# Binary point is part of the solution, but not an entire encoding

- Some problems remain:
  1. Computers are finite, but real numbers are not
    - Need to choose how many bits to use
    - Many decimal numbers would take infinite binary bits to represent perfectly
      - $3.14_{10} = 11.0010001111010111_2$  (we could keep going)
  2. We also need to represent where the “binary point” is located
    - We’ll use scientific notation and some of our bits to do so
  3. Should include signed numbers while we’re at it

# Topics

- Integer Operations
- Binary Operations – Boolean Algebra
- Bit Shifting
- Bit Masking
- **Floating Point**
  - Fractional binary numbers
  - **Representing floating point**

# Floating Point Standard – IEEE754

[https://en.wikipedia.org/wiki/IEEE\\_754](https://en.wikipedia.org/wiki/IEEE_754)

- Floating point representations
  - Encodes rational numbers of the form  $V = m \times 2^e$
  - Base 2 scientific notation!
- IEEE Standard 754 (IEEE floating point)
  - Established in 1985 as uniform standard for floating point arithmetic
    - Before that, many various formats with different rules
  - Headed by William Kahan, CS prof. at UC Berkeley (later won Turing Award for it)
  - Supported by all major CPUs
- Driven by numerical concerns and numerical analysts
  - Nice standards for rounding, overflow, underflow
  - Had to be implementable in fast hardware as well and support many languages

# Floating Point Representation

Numerical form:

$$V = (-1)^S * M * 2^E$$

Sign bit      Significand (Mantissa)      Exponent

- Sign bit **S** determines whether number is negative (1) or positive (0)
- Significand **M** normally a fractional value in range [1.0,2.0) or [0.0,1.0)
  - Called ***mantissa*** or ***significand***
- Exponent **E** weights value by power of two
- Heads up: Most of this equation is decimal, except Significand, which is floating point binary

# IEEE754 Floating Point Encoding

Numerical form:

$$V = (-1)^s * M * 2^E$$

Diagram illustrating the numerical form of a floating-point number  $V$ :

- $s$ : Sign bit (indicated by a blue arrow labeled "Sign bit")
- $M$ : Significand (Mantissa) (indicated by a blue arrow labeled "Significand (Mantissa)")
- $E$ : Exponent (indicated by a blue arrow labeled "Exponent")

- IEEE754 Encoding
  - MSb is sign bit (can still look at most-significant bit alone to determine sign!)
  - **exp** field encodes  $E$ ,  $k$ -bits (note: "*encodes  $E$* " != "*is  $E$* ")
  - **frac** field encodes  $M$ ,  $n$ -bits



# IEEE754 Floating Point Precision

- Sizes

- Single precision:  $k = 8$  exp bits,  $n = 23$  frac bits (32b total). **float** in C



- Double precision:  $k = 11$  exp bits,  $n = 52$  frac bits (64b total). **double** in C



# Categories for IEEE754 Encoded Values

Value encoded – three cases, depending on value of **exp**

1. Normalized, the most common



2. Denormalized (very small values)



3. Special values – infinity and NaN



# Categories for IEEE754 Encoded Values

Value encoded – three cases, depending on value of **exp**

## 1. Normalized, the most common



## 2. Denormalized (very small values)



## 3. Special values – infinity and NaN



# Normalized, Significand

$$V = (-1)^s * \boxed{M} * 2^E$$

- Condition: not a special exponent (all zeros or ones)
- Significand (M) is encoded with implied leading 1
  - $M = 1.xxx...x_2$  (implied leading 1 representation)
    - xxx...x: bits of **frac** used directly (i.e., 1.**frac**)
    - **frac** ranges from [0,1)
- Idea: every normalized number is 1.xxxx
  - So we're not going to include the leading 1 in the frac
  - We'll just know it's there when we convert to decimal
  - Saves one extra bit in the encoding!



$$V = (-1)^s * M * 2^E$$

# Normalized, Exponent

- Condition: not a special exponent (all zeros or ones)
- Exponent coded as a biased value
  - $E = \text{Exp} - \text{Bias}$ 
    - Exp : unsigned value denoted by **exp**
    - Bias : Bias value =  $2^{k-1} - 1$ ,  $k$  is number of exponent bits
      - Single precision (8-bit exp): 127 (Exp: 1...254, E: -126...127)
      - Double precision (11-bit exp): 1023 (Exp: 1...2046, E: -1022...1023)
- Exponent really just pushes the binary point around
  - $1.11 * 2^2 = 11.1 * 2^1 = 111.0 * 2^0 = 111$
  - $111 * 2^{-2} = 11.1 * 2^{-1} = 1.11 * 2^0 = 1.11$



# Decoding example for normalized floating point (32-bit)

- $0xC1900000 = 0b11000001100100000000000000000000$ 
  - Group bits **s**: 1 **exp**: 10000011 **frac**: 001000000000000000000000
  - **exp** is not all zeros or all ones  $\Rightarrow$  not a special case
- $M = 1.001000000000000000000000 = 1.001$
- $\text{exp} = 10000011 = 131$ 
  - $E = \text{exp} - \text{bias} = 131 - 127 = 4$ 
    - $\text{bias} = 2^{k-1} - 1, k=8 \rightarrow 2^7 - 1 = 127$
- $\text{Result} = (-1)^1 * 1.001_2 * 2^4 = -10.01_2 * 2^3 = -10010.2 = -18$

|                        |                                                                        |      |     |      |
|------------------------|------------------------------------------------------------------------|------|-----|------|
| $V = (-1)^s * M * 2^E$ | <table border="1"><tr><td>s</td><td>exp</td><td>frac</td></tr></table> | s    | exp | frac |
| s                      | exp                                                                    | frac |     |      |

# Normalized Encoding Example

- **Value**

- `float F = 15213.0; // single precision: 8 exp bits, 23 frac bits`
- $15213_{10} = 11101101101101_2 = 1.1101101101101_2 \times 2^{13}$

- **Significand**

- $M = 1.\underline{1101101101101}_2$
- $\text{frac} = \underline{1101101101101}0000000000$  pad with 0s *on the right*. (example: 1.5 = 1.500)

- **Exponent**

- $E = 13$
- Bias = 127
- $\text{exp} = E + \text{Bias} = 140 = 10001100_2$

**Floating Point Representation:**

|         |      |      |      |      |      |      |      |           |
|---------|------|------|------|------|------|------|------|-----------|
| Hex:    | 4    | 6    | 6    | D    | B    | 4    | 0    | 0         |
| Binary: | 0100 | 0110 | 0110 | 1101 | 1011 | 0100 | 0000 | 0000      |
| exp:    | 100  | 0110 | 0    |      |      |      |      |           |
| frac:   |      |      |      | 110  | 1101 | 1011 | 0100 | 0000 0000 |

# Normalized Numbers: Why These Choices?

- Significand coded with implied leading 1
  - Any non-zero integer will start with a 1 bit somewhere
  - Leading 1 carries no information, so don't need to store it!
  - Can express mantissas between:
    - 1.0 when frac is all 0s
    - 2.0 (nearly) when frac is all 1s
      - Want smaller? Use a smaller exponent!
- Exponent coded as biased value
  - $E = Exp - Bias$
  - Alternative to using two's complement to represent signed integers
  - Reasons are a bit tricky
    - Floating point binary values increase in the same order as unsigned, which means they can be compared just like integer encodings
    - Bias also provides a more useful range (when considering denormalized)

# Example

- $0x3F800000 = 0b00111111100000000000000000000000$ 
  - Group bits **s**: 0 **exp**: 01111111 **frac**: 00000000000000000000000000000000
  - **exp** is not 0...0 or 1...1  $\Rightarrow$  not a special case
- $M =$
- $E = \mathbf{exp} - \mathbf{bias} =$ 
  - $\mathbf{bias} = 2^{k-1} - 1, k=8 \rightarrow 2^7 - 1 = 127$

$$V = (-1)^s * M * 2^E$$



# Example

- $0x3F800000 = 0b00111111100000000000000000000000$ 
  - Group bits **s**: 0 **exp**: 01111111 **frac**: 00000000000000000000000000000000
  - **exp** is not 0...0 or 1...1  $\Rightarrow$  not a special case
- $M = 1.00000000000000000000000000000000 = 1.0$
- $E = \mathbf{exp} - \mathbf{bias} = 127 - 127 = 0$ 
  - $\text{bias} = 2^{k-1} - 1, k=8 \rightarrow 2^7 - 1 = 127$
- $\text{Result} = (-1)^0 * 1.0_2 * 2^0 = 1$

$$V = (-1)^s * M * 2^E$$



# Categories for IEEE754 Encoded Values

- Value encoded – three cases, depending on value of **exp**

1. Normalized, the most common



2. Denormalized (very small values)

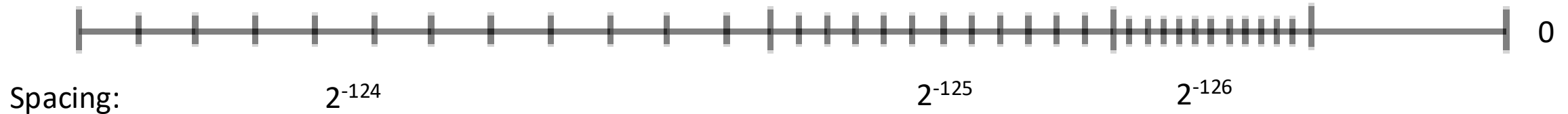


3. Special values – infinity and NaN

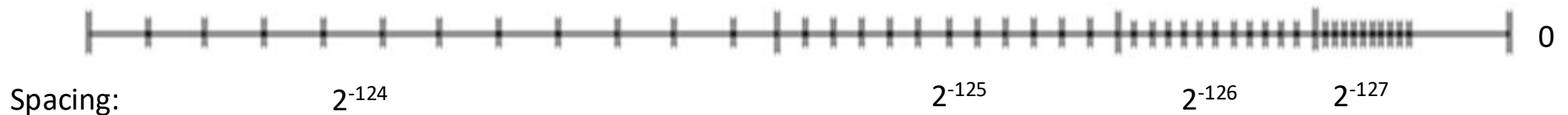


# Normalized floating point leaves a gap around zero

- Gap is the size of  $1.0000 * 2^{\text{Min Exponent}}$  (due to leading 1 bit)
  - How do we encode “zero?”

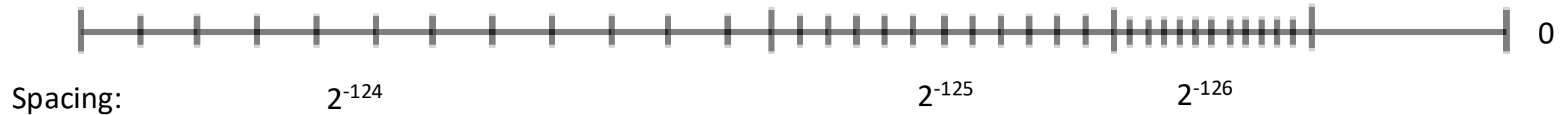


- Problem: if we just kept doing what we're doing we never reach 0
  - We keep getting half-way there



# Solution is to do something different for the smallest numbers

- Problem: always starting with 1.XXXXX leaves a gap



- Solution: fill in numbers between 0 and  $1 * 2^{\text{Min Exponent}}$ 
  - Using same spacing as the previous range, in the form 0.XXXXX
  - **Gradual underflow** – possible numeric values are spaced evenly near 0.0



$$V = (-1)^s * M * 2^E$$

# Denormalized Values

- Purpose: gracefully represent numbers approaching  $\pm 0$
- Condition:  $\text{exp} = 000\dots 0_2$
- Value
  - Exponent value  $E = \underline{1 - \text{Bias}}$ 
    - Note: NOT  $E = 0 - \text{Bias}$  as it would be if we followed the previous rules
    - This means we're instead re-using the spacing from smallest normalized numbers
  - Significand value  $M = \underline{0}.\text{xxx}\dots\text{x}_2$  ( $0.\text{frac}$ )
    - $\text{xxx}\dots\text{x}$ : bits of  $\text{frac}$ . Leading 0 instead of leading 1
- Cases
  - $\text{exp} = 000\dots 0$ ,  $\text{frac} = 000\dots 0 \Rightarrow$  Represents value 0
    - Note that we have distinct values  $+0$  and  $-0$ , which depends on if the sign bit is 0 or 1, respectively
  - $\text{exp} = 000\dots 0$ ,  $\text{frac} \neq 000\dots 0 \Rightarrow$  Numbers very close to 0.0

# Categories for IEEE754 Encoded Values

- Value encoded – three cases, depending on value of **exp**

1. Normalized, the most common



2. Denormalized (very small values)



3. Special values – infinity and NaN



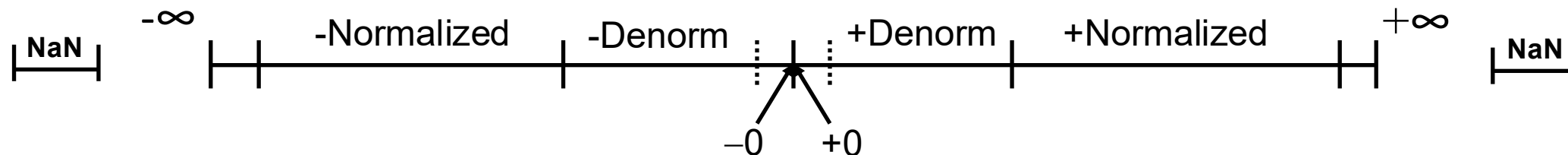
# Special Values

- Purpose: represent quantities that  $(-1)^s * M * 2^E$  cannot
- Condition:  $\text{exp} = 111\dots 1_2$
- Cases
  - $\text{exp} = 111\dots 1_2, \text{frac} = 000\dots 0_2$ 
    - Represents value  $\infty$  (infinity)
    - Both positive and negative infinity (sign bit to tell apart,  $s=0$  is positive and  $s=1$  is negative)
    - Operation that overflows: nicer mathematical behavior than truncation/modulo when numbers get too big or small!
    - E.g.,  $1.0/0.0 = -1.0/-0.0 = +\infty$ ,  $-1.0/0.0 = -\infty$ 
      - IEEE 754 has a divide by zero exception that prevents programs from crashing since default behavior is to produce an infinity value and set a floating-point status flag
  - $\text{exp} = 111\dots 1_2, \text{frac} \neq 000\dots 0_2$ 
    - Not-a-Number (NaN)
    - Represents case when no numeric value can be determined
      - Fraction could be used to distinguish sources (rarely used in practice)
    - E.g.,  $\sqrt{-1}$ ,  $\infty - \infty$ ,  $\infty * 0$

# Floating Point in C

- C guarantees two levels
  - **float** single precision
  - **double** double precision
- Conversions
  - **int** → **float**
    - maybe rounded
    - less bits for actual value (32 → 23)
  - **int** or **float** → **double**
    - exact value preserved
    - double has greater range and higher precision (52 bits for **frac**)
  - **double** → **float**
    - may overflow, underflow (too small to represent), or be rounded (IEEE 754)
    - C99 standard says **undefined** if value out of range
  - **double** or **float** → **int**
    - rounded toward zero (-1.999 → -1)
    - C99 standard says **undefined** if value out of range

# Summary of FP Real Number Encodings



$$V = (-1)^s * M * 2^E$$

|       | Normalized                                                            | Denormalized                                                 |
|-------|-----------------------------------------------------------------------|--------------------------------------------------------------|
| s     | 0/1 means +/-                                                         | 0/1 means +/-                                                |
| exp   | $\text{exp} \neq 000\dots 0_2$ and $\text{exp} \neq 111\dots 1_2$     | $\text{exp} = 000\dots 0_2$                                  |
| frac  | $x_1x_2x_3\dots x_j$                                                  | $x_1x_2x_3\dots x_j$                                         |
| Bias= | $2^{(k-1)} - 1$ , for $k$ exponent bits                               | $2^{(k-1)} - 1$ , for $k$ exponent bits                      |
| E=    | $\text{exp} - \text{Bias}$                                            | $1 - \text{Bias}$                                            |
| M=    | $1.x_1x_2x_3\dots x_j$ a.k.a. 1.frac                                  | $0.x_1x_2x_3\dots x_j$ a.k.a. 0.frac                         |
| V=    | $(-1)^s \times (1.\text{frac}) \times 2^{(\text{exp} - \text{Bias})}$ | $(-1)^s \times (0.\text{frac}) \times 2^{(1 - \text{Bias})}$ |